

Adding Planned Design to XP Might Help Novices' Productivity (or Might Not): Two Controlled Experiments

René Noël, Gonzalo Valdes, Marcello Visconti, Hernán Astudillo

Universidad Técnica Federico Santa María

Av. España 1680, Valparaíso, Chile

{rnoel, gvaldes, visconti, hernan}@inf.utfsm.cl

ABSTRACT

Extreme Programming (XP) suggests using Evolutionary design, always implementing the simplest solution that satisfies the current iteration's requirements, instead of Planned (or Traditional) up-front design. Some developers have questioned the usefulness of Evolutionary approach's enabling practices (e.g., refactoring, test-driven development) arguing for the naturalness of, and need for, Planned design. Two controlled experiments were conducted to compare both approaches regarding product quality and programmer productivity. Results from both studies show that (1) there is no significant difference in the product quality, independently of experience, but (2) novices are more productive using the Planned approach.

Categories and Subject Descriptors

D.2.4 [Software Engineering]: Software/Program Verification – Statistical Methods.

General Terms

Design, Experimentation.

Keywords

Extreme Programming, Evolutionary Software Design, Planned Software Design, Experimental Studies.

1. INTRODUCTION

Extreme Programming (XP) proposes an iterative process and an Evolutionary design approach that encourages implementing the simplest solution for the requirements considered in the current iteration, without worrying about next iterations' requirements and their design complexity [1]. This approach allows embracing change by avoiding big up-front design activities and tackles overall software structural complexity only when needed, as opposed to Planned (or Traditional) design [6]. In this study Design is understood as a set of low-level software design activities (e.g., class models design) and does not comprehend any architectural design issues.

There are documented cases of development teams that resist to

this approach (academic environment in [3] and [4], and practitioners environment in [2] and [7]), questioning XP design practices and their naturalness; also, a big up-front design can be an instance for identifying reusable structures (such as design patterns) that might help improving productivity and product internal quality.

This paper presents the results of two controlled experiments conducted to evaluate the impact of these two design approaches on the resulting product design quality and development process productivity (both in academic environments). Section 2 defines the experimental study, section 3 describes the first experiment and its results, section 4 describes the second experiment, and section 5 states the main conclusions.

2. DEFINING THE EXPERIMENTAL STUDY

2.1 Experimental Activity Design

The software development activity that subjects must tackle consisted of designing, coding and testing software for a given problem using either design approach (XP/Evolutionary or Planned). The solution should be implemented under the Object Oriented Paradigm (using the Java language) during two development iterations; trials could not take more than four hours.

When applying each design approach all XP practices that provide an instance to think about design must be considered, thus two development methods were defined: one based on the XP original definition [1] (called *Method 1*); the other as a variation of the original XP process (called *Method 2*), which incarnates the Planned design approach by incorporating a planned design session at the start of each iteration.

2.2 Hypotheses, Variables and Metrics

High level null hypotheses are:

H0': The use of a planned design approach produces software with an equal design quality than evolutionary design approach.

H0'': The use of an evolutionary design approach is as productive as a planned design approach in an XP project.

Independent variables that can affect quality and productivity are: the *Design Approach*, which is the factor under study, with two levels (XP and Planned); the *Problem to Solve*, which is the experimental object, i.e. the programming problem that subjects had to solve applying the design approaches; and two undesirable factors of influence, *Participants' XP Knowledge* and *Participants' Design Experience*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ESEM'08, October 9–10, 2008, Kaiserslautern, Germany.

Copyright 2008 ACM 978-1-59593-971-5/08/10...\$5.00.

The quality concept applied in this study is that a good design should keep low levels of algorithm's complexity, measured with the metrics: Decision Count (DC), Maximum McCabe Cyclomatic Complexity (MCC), and Number of Code Statements (NSTMNT). For analyzing variations in productivity, three metrics were chosen to capture different views of the effort put into the design, namely: Number of Classes/hour (PTNOC), LOC/minute (PTLOC), and Number of Methods/minute (PTNOM).

3. THE FIRST EXPERIMENT

3.1 Experimental Design

The first experiment has a random blocked design, with one factor, the Design Approach. Participants were undergraduate computer science students that had previously approved an Object Oriented Analysis and Design course and had backgrounds in OO programming, Software Engineering and Software Design Patterns. Subjects were 31 development teams, each one being comprised by a pair of participants with equivalent design experience (determinate through a survey). Subjects were divided in two blocks: subjects with design experience and subjects without design experience (marked with two different grey tones in Table 1).

Table 1. First Experimental Design.

Subject	Method 1	Method 2
1	X	
2		X
...		
30	X	
31		X

The Problem to solve presents a business logic that forces subjects to decide where to place the complexity: on the classes design or on the algorithms within classes' methods. The Problem's requirements were packed into two sets, the sets conformation was design so the second set induced refactoring opportunities to the code that was implemented for satisfying the first set.

3.2 Results

Table 2 shows the resulting p-values of a two-sided t-Test of difference between means for design experienced subjects (DE) and for non-experienced subjects (NDE). The PTNOC metric with NDE subjects was the most significant result ($p \sim 0.05$), so a one-sided t-Test to check if Method 2 is more productive than Method 1 is applied, PTLOC resulting p-value is $p = 0.0260$ ($p < 0.05$). This means that the Planned approach helped non-experienced subjects to produced more classes, an interpretation is that they made an effort for placing the design complexity into the classes structure rather than mere placing it into the methods' classes algorithms (if this would be the case then only the PTLOC metric would show significant differences); unfortunately none of the metrics used for software design quality showed significant differences, so we can not state that quality was actually improved with the Planned approach.

The difference between experience blocks leads us to try to mitigate the influence of this factor instead of blocking it, in order to isolate the method effect from the experience effect. Thus we conducted a second experiment with a different experimental design, which is described in the next section.

Table 2. First Experiment, p-value for each metric.

	Productivity			Quality		
	PTNOC	PTLOC	PTNOM	DC	NSTMNT	MCC
NDE	0.0520	0.0804	0.1503	0.3726	0.4424	0.5843
DE	0.8078	0.1481	0.6047	0.2281	0.2645	0.4859

4. THE SECOND EXPERIMENT

4.1 Experimental Design

We used a 2x2 factorial design with repeated measures; factors are the Design Approach and the Problem to Solve. Participants were 22 senior-level computer science students, with Object Oriented Design knowledge and low levels of professional practice (in average they were two years older than the first experiment participants and were following the same academic program). Each subject is comprised by two participants (same as first experiment, but without worrying about matching their experience level).

Table 3 presents the second experimental design. In this design subjects were asked to participate twice (instead of once, which was the case for the first experiment). Subjects were divided in two groups randomly, during the first execution half of Group 1 applied Method 1 on Problem 1 and the other half applied Method 2 on Problem 2, methods and problems were inverted during the second execution; similar strategy was applied for Group 2 (see Table 3). The problems to solve are equivalents with the problem from the first experiment.

Table 3. Second Experimental Design.

	Group 1	Group 2
Method 1	Training	Training
	Problem 1	Problem 2
Method 2	Training	Training
	Problem 2	Problem 1

4.2 Results

The data was analyzed applying an Analysis of Variance of the 2x2 factorial design with repeated measures. Looking at method's effect (design's approach effect) in Table 4, we can see that the null hypotheses can not be rejected; thus no differences in quality and productivity between Evolutionary and Planned design approaches can be demonstrated by the influence of the Method factor.

A less formal analysis can be made looking at the methods' estimated marginal means for each metric; a summary is presented in Table 5. Marginal means suggest that (although not statistically significant) the Planned Approach (Method 2) was

more productive than the XP Approach (Method 1) for every metric of productivity, i.e., the XP Approach produced less LOC, classes and methods.

Table 4. Second Experiment, ANOVA Test Results for the Method's Effect.

Factor	Metric	Degrees of freedom	Sum of squares	Mean square	F-value	p-value
Method	PTNOC	1	0.316	0.316	0.781	0.403
	PTLOC	1	0.108	0.108	2.754	0.136
	PTNOM	1	0.001	0.001	0.122	0.736
	DC	1	2.78	2.78	0.426	0.532
	NSTMNT	1	177.78	177.78	1.171	0.311
	MCC	1	11.11	11.11	1.413	0.269

Product design quality metrics are favorable to XP Design Approach for Number of Statements and Maximum Cyclomatic Complexity, Decision Count metric suggests that more control flow statements were coded when using XP Approach than using the Planned Approach, i.e., XP Approach produced better code in two (out of 3) metrics.

This might mean that the XP approach produced less but better code. Unfortunately none of these conjectures could be formally demonstrated, because results were not statistically significant.

Table 5. Estimated Marginal Means for the Method's Effect.

Metric	Method	Mean	Standard Error
PTNOC	1	1.106	0.162
	2	1.294	0.133
PTLOC	1	0.458	0.053
	2	0.568	0.080
PTNOM	1	0.079	0.020
	2	0.090	0.025
DC	1	3.611	0.740
	2	3.056	0.868
NSTMNT	1	22.167	2.26
	2	26.611	3.76
MCC	1	3.333	0.717
	2	4.444	0.966

More detail about the results and the instrumentation can be found in the studies lab package [5].

5. CONCLUSIONS

During the first experiment the results showed significant differences for NDE subjects with one productivity metric, i.e., NDE subjects using the Planned approach produced more classes.

This might lead to better quality code (placing design complexity in the classes structure rather than in the methods' algorithms), but unfortunately none of the quality metrics showed statistically significant differences. All results for DE subjects were non-significant.

During the second experiment we sought to mitigate the design experience factor by changing the experimental design, we no longer divided them into experience groups. The results did not indicate any significant differences, and we could only make the following less formal conjectures: marginal means suggest that the XP approach produced less code than the Planned approach, and two (out of 3) quality metrics showed that the XP approach produce better quality code. This might mean that XP approach produced less but better code (again this is only a conjecture).

Results from both studies lead us to believe that if there are significant differences in productivity and quality, then they should manifest themselves more strongly in non-experienced developers, to whom a Planned approach would help more than it would help experience developers. Some threats to this conjecture are that the problems given to subjects to apply the design approaches were too small and were insufficient to elucidate significant differences, and that the subjects grouping (experienced and not experienced) that we made were not rigorous enough. Also, using other criteria to separate them could alter the results.

A main conclusion is that, when facing the design activity, we should evaluate the trade-offs between increasing process productivity through a Planned design approach, and empowering the process capability for embracing change through XP original design approach, without premeditatedly affecting product design quality.

6. REFERENCES

- [1] Beck, K. 1999. Extreme Programming Explained: Embrace Change. Addison-Wesley.
- [2] Harrison, N. 2003. A Study of Extreme Programming in a Large Company. Avaya Labs. <http://www.agilealliance.org/system/article/file/1292/file.pdf>
- [3] Keefe, K. and Dick, M. 2004. Using Extreme Programming in a Capstone Project. Proc. 6th Australasian Computing Education Conf. (Dunedin, New Zealand, 2004), 151-160.
- [4] Müller, M. and Tichy, W. 2001. Case Study: Extreme Programming in a University Environment. Proc. 23rd Int'l Conf. on Software Eng. (Toronto, Canada, 2001), 537-544.
- [5] Noël, R., Visconti, M., Valdes, G. and Astudillo, H. 2007. Lab. Package for the Investigation about the Impact of Software Design Approaches on XP. http://www.labada.inf.utfsm.cl/~amigosisw/xpdesign_package
- [6] Pressman, R. 2004. Software Engineering: A Practitioner's Approach. McGraw-Hill, 6th edition.
- [7] Rasmusson, J. 2003. Introducing XP into Greenfield Projects: Lessons Learned. IEEE Software, vol. 33, no. 7, 21-28.